

APPENDIX

```
;; value-of : Exp * Env -> ExpVal
(define value-of
  (lambda (exp env)
    (cases expression exp

      (const-exp (num) (num-val num))

      (var-exp (var) (apply-env env var))

      (diff-exp (exp1 exp2)
        (let ((val1 (value-of exp1 env))
              (val2 (value-of exp2 env)))
          (let ((num1 (expval->num val1))
                (num2 (expval->num val2)))
            (num-val
             (- num1 num2))))))

      (zero?-exp (exp1)
        (let ((val1 (value-of exp1 env)))
          (let ((num1 (expval->num val1)))
            (if (zero? num1)
                (bool-val #t)
                (bool-val #f)))))

      (if-exp (exp1 exp2 exp3)
        (let ((val1 (value-of exp1 env)))
          (if (expval->bool val1)
              (value-of exp2 env)
              (value-of exp3 env))))

      (let-exp (var exp1 body)
        (let ((val1 (value-of exp1 env)))
          (value-of body
                     (extend-env var val1 env))))

      (proc-exp (var body)
        (proc-val (procedure var body env)))

      (call-exp (rator rand)
        (let ((proc (expval->proc
                          (value-of rator env)))
              (arg (value-of rand env)))
          (apply-procedure proc arg)))

      )))

(define apply-procedure
  (lambda (proc1 val)
    (cases proc proc1
      (procedure (var body saved-env)
        (value-of body
                    (extend-env var val saved-env))))))
```

```
(module lang (lib "eopl.ss" "eopl")

  ;; grammar for the PROC language

  (require "drscheme-init.scm")

  (provide (all-defined))

  ;;;;;;;;;;; grammatical specification ;;;;;;;;;;;

  (define the-lexical-spec
    '((whitespace (whitespace) skip)
      (comment ("% (arbno (not #\newline))) skip)
      (identifier
        (letter (arbno (or letter digit "_" "-" "?")))
        symbol)
      (number (digit (arbno digit)) number)
      (number ("-" digit (arbno digit)) number)
    ))

  (define the-grammar
    '((program (expression) a-program)

      (expression (number) const-exp)

      (expression
        ("-" "(" expression "," expression ")")
        diff-exp)

      (expression
        ("zero?" "(" expression ")")
        zero?-exp)

      (expression
        ("if" expression "then" expression "else" expression)
        if-exp)

      (expression (identifier) var-exp)

      (expression
        ("let" identifier "=" expression "in" expression)
        let-exp)

      (expression
        ("proc" "(" identifier ")" expression)
        proc-exp)

      (expression
        "(" expression expression ")"
        call-exp)

    ))
```